Evaluating Membership and Broadcast Protocol Combinations in Gossip-Based Systems

Miguel Mestre*
mfg.mestre@campus.fct.unl.pt
MEI, DI, FCT, UNL

Alexandre Santos†
aff.santos@campus.fct.unl.pt
MEI, DI, FCT, UNL

Nelson‡
marbles@cool.se
MEI, DI, FCT, UNL

Abstract

Gossip-based protocols have emerged as an effective approach to implement scalable and resilient broadcast mechanisms in distributed systems. In such systems, nodes rely on membership protocols to maintain partial views of the network, which directly influence the efficiency and reliability of message dissemination. The performance of gossip-based broadcast depends on the interaction between membership and broadcast strategies, particularly under varying system conditions.

Several approaches can be used to construct and maintain these overlays, each offering different trade-offs in terms of reliability, latency, and communication overhead. In this paper, we implement and experimentally evaluate combinations of membership protocols, namely Cyclon and HyParView, with broadcast protocols such as Flood and Eager-Push Gossip, as well as a custom-designed gossip-based solution. Our results show that different protocol combinations exhibit distinct performance characteristics, highlighting trade-offs between reliability and latency, and demonstrating that the choice of membership protocol plays a key role in determining the effectiveness of broadcast mechanisms.

ACM Reference Format:
Miguel Mestre, Alexandre Santos, and Nelson. 2022. Evaluating Membership and Broadcast Protocol Combinations in Gossip-Based Systems. In *The Projects of DistAlg - first project, 2026, Faculdade de Ciências e Tecnologia, NOVA University of Lisbon, Portugal*. ACM, New York, NY, USA, **??** pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Modern distributed systems often need to send a message to all nodes participating in the system, an operation known as broadcast. The simplest way to do this is flooding, where every node that receives a message immediately forwards it to all of its neighbors. This guarantees that every node will eventually receive the message, but it comes at a steep cost. Because every node forwards to every neighbor, the total number of messages in the network grows very quickly as the system scales, filling up network links and forcing each node to spend time processing and discarding large numbers of duplicate messages. This extra work does not go unnoticed: the congestion and processing overhead introduced by flooding ultimately manifests as increased end-to-end latency, meaning messages take longer to arrive not because the network is slow, but because it is overloaded.

A more efficient alternative is gossip-based broadcast, where instead of forwarding to every neighbor, each node picks only a small random subset of neighbors to forward to. The size of this subset is controlled by a parameter called the fanout. At first glance it might seem that reducing the number of forwarding targets would significantly hurt reliability, but in practice, if the fanout is well calibrated, the message still reaches the vast majority of nodes in the system. The core argument is therefore that the overhead flooding introduces does not meaningfully improve reliability over a well tuned gossip protocol, it only adds congestion and latency.

For gossip to work well, nodes need a way to know which other nodes exist in the system and to maintain a set of neighbors to gossip with. This is the role of an overlay network protocol. In this work we consider two such protocols: Cyclon and HyParView. Cyclon maintains a single list of neighbors and periodically shuffles it with other nodes to keep the view random and fresh. It is simple and works well in stable conditions. HyParView takes a different approach and maintains two lists: an active view of neighbors that are currently in use, and a passive view of backup neighbors that are kept in reserve. When an active neighbor leaves or stops responding, HyParView can immediately replace it with a node from the passive view without waiting for the next maintenance cycle. This makes HyParView more resilient when nodes frequently join and leave the network, a condition known as churn. In scenarios with little churn however, the simpler design of Cyclon is expected to introduce slightly less overhead and therefore slightly lower latency.

On top of these overlay protocols we implement a hybrid gossip broadcast protocol. Rather than always forwarding the full message to neighbors, our protocol splits neighbors into two groups per message. A small subset called eager targets receives the full message right away. Another subset called lazy targets receives only a small announcement indicating that the sender has a message available. If a lazy target has not yet seen that message, it can request the full content, effectively pulling it on demand. This reduces unnecessary full message transmissions when nodes have already received the message through another path. Additionally, our protocol monitors how many duplicate messages it is receiving and uses this information to dynamically adjust the eager fanout over time, growing it when too few nodes seem to be receiving messages and shrinking it when too many duplicates are observed. We expect this adaptive behavior to reduce message overhead compared to a static eager

*Student number 73018
†Student number 72970
‡Student number 11111. Marbles did not understood the assignment but he did use an LLM to make a song related with the project.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
DistAlg25/26, Faculdade de Ciências e Tecnologia, NOVA University of Lisbon, Portugal
© 2022 ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

push approach, though the dynamic adjustment of the fanout may occasionally cause short periods of reduced delivery reliability.

In this paper we experimentally study these three aspects: whether flooding truly introduces more latency than gossip for similar reliability levels, whether HyParView outperforms Cyclon under high churn while Cyclon holds an edge in stable networks, and whether our hybrid adaptive protocol improves on static gossip in terms of overhead while remaining competitive in reliability. The remainder of this paper is organized as follows. Section **??** discusses related work on overlay networks and gossip protocols. Section **??** describes our implementation in detail. Section **??** presents and discusses our experimental results. Finally, Section **??** concludes the paper.

2 Related Work

This section reviews prior work that is necessary to contextualize the protocol combinations studied in this paper. We first discuss random overlay construction protocols, then gossip-based broadcast protocols, and finally prior experimental studies that compare dissemination strategies in similar settings.

2.1 Random Overlay Networks

Random overlay networks are a common building block for scalable distributed systems because they provide each node with only a partial view of the network while still preserving good connectivity and short communication paths. Instead of requiring every node to know all other participants, these protocols maintain a small and continuously refreshed neighbor set, which is enough to support robust dissemination and membership management at scale.

Cyclon is one of the best-known peer-sampling protocols in this space. Its main goal is to maintain a continuously evolving random overlay by periodically exchanging small neighbor samples between nodes. This periodic shuffling process keeps views fresh, removes stale entries, and helps preserve overlay randomness over time. Because of its simple design, Cyclon is often used as a baseline membership mechanism in experimental studies of gossip-based systems.

HyParView was proposed to improve robustness under churn while still maintaining a lightweight overlay. In contrast with protocols that rely on a single partial view, HyParView separates neighbor management into two structures: a small active view used for communication and a larger passive view used as backup. This distinction allows the protocol to react quickly to failures by replacing lost active neighbors from the passive view, while periodic maintenance keeps the passive view updated. As a result, HyParView is generally regarded as better suited to highly dynamic environments.

These two protocols represent different design choices in overlay construction. Cyclon emphasizes simplicity and randomness through periodic exchanges, whereas HyParView combines reactive and periodic mechanisms to improve failure resilience. This makes them natural candidates for comparison when studying how the choice of membership layer affects broadcast performance.

2.2 Gossip-Based Broadcast

Gossip-based broadcast protocols were introduced as a scalable alternative to deterministic flooding. In flooding, each node forwards a received message to all its neighbors. This strategy is simple and achieves full dissemination in connected overlays, but it generates a large number of duplicate transmissions and places unnecessary load on the network. As the system grows, this redundancy increases overhead and may also increase end-to-end latency due to congestion and repeated processing.

Eager-push gossip reduces this cost by forwarding each message only to a subset of neighbors, usually chosen at random. The size of this subset, commonly called the fanout, determines the trade-off between reliability and overhead. When the fanout is too small, messages may fail to reach the whole system. When it is too large, the protocol starts behaving closer to flooding and produces many duplicates. A well-chosen fanout therefore allows gossip-based broadcast to retain high coverage while significantly reducing communication costs.

Several works also explore hybrid dissemination strategies. Instead of always sending full payloads, some protocols first advertise the existence of a message and only transmit the full content when needed. This idea appears in lazy-push and pull-based designs, where message identifiers are disseminated separately from message bodies. Such approaches can reduce redundant full-message transmissions, especially when many nodes have already received the same content through different paths.

Plumtree is one of the best-known protocols combining eager and lazy strategies. Its design attempts to exploit the low latency of tree-based dissemination while preserving the resilience of gossip through a backup mechanism. This line of work shows that combining dissemination modes can provide better trade-offs than purely eager or purely lazy schemes.

Our hybrid protocol is inspired by this family of approaches. However, instead of fixing the balance between eager and lazy dissemination once and for all, we adapt the eager fanout dynamically according to the observed duplication rate. This design choice aims to let the protocol react to the conditions of the overlay rather than relying on a single static parameter.

2.3 Experimental Studies of Gossip Protocols

A substantial body of prior work evaluates gossip-based dissemination protocols experimentally, typically focusing on metrics such as reliability, dissemination latency, message overhead, and robustness under churn. These studies repeatedly show that dissemination behavior cannot be analyzed independently from overlay construction. The quality of the membership layer influences path diversity, redundancy, and the ability to recover from failures, all of which affect broadcast performance.

Prior experimental comparisons also show that flooding tends to achieve high reliability at the cost of excessive overhead, whereas probabilistic gossip often achieves comparable coverage with fewer transmissions when its parameters are well tuned. Similarly, overlays designed for dynamic environments tend to outperform simpler overlays under churn, even if they may introduce some additional maintenance cost in stable scenarios.

The goal of our work is aligned with this experimental tradition. Rather than proposing a purely theoretical argument, we implement different combinations of membership and broadcast protocols and analyze their behavior under common dissemination metrics. In

this sense, our study is not only about the protocols in isolation, but also about the interaction between overlay maintenance and message dissemination.

3 Implementation

This section describes the implementation of the system developed for this project. The goal of the implementation was to make it possible to experimentally compare multiple combinations of membership and broadcast protocols within a common execution framework. To achieve this, we organized the system into two main layers: a membership layer, which is responsible for maintaining the overlay network, and a broadcast layer, which uses that overlay to disseminate messages. We first present the overall organization of the implementation, then describe the membership protocols, and finally discuss the broadcast protocols and the custom hybrid solution.

3.1 System Organization

The system was implemented using the Babel framework, which supports the composition of protocols through message handlers, timers, channel events, and inter-protocol notifications. This model was particularly useful for the project because it allowed us to separate membership logic from broadcast logic while still enabling communication between both layers.

The membership protocols expose changes in the local neighborhood through notifications that inform the broadcast layer when neighbors become available or unavailable. This design keeps the broadcast protocols independent from the internal details of overlay maintenance and makes it possible to combine the same broadcast algorithm with different membership mechanisms.

At a high level, every node executes one membership protocol and one broadcast protocol. The membership protocol builds and maintains the local partial view of neighbors. The broadcast protocol relies on that view to forward messages. This separation mirrors the conceptual structure of the project and simplifies both experimentation and code reuse.

3.2 Membership Layer

3.2.1 Cyclon. Our Cyclon implementation follows the standard peer-sampling approach based on partial views and periodic shuffling. Each node maintains a view containing neighbor entries associated with an age. At regular intervals, the node increments the age of its entries, selects one neighbor, and exchanges a sample of entries with that node. This process gradually refreshes the view and helps preserve randomness in the overlay.

The main advantage of Cyclon in our implementation is its simplicity. Since all view maintenance is driven by periodic exchanges, the protocol is compact and easy to integrate with the rest of the system. However, this same design means that recovery from failures is less immediate, because failed nodes are only removed or replaced when subsequent maintenance actions occur.

3.2.2 HyParView. Our HyParView implementation extends the simpler peer-sampling approach by maintaining two views: an active view and a passive view. The active view contains the neighbors currently used for communication, while the passive view contains backup nodes that can be promoted when necessary.

When a node joins the system, it contacts a known participant and sends a join request. The contacted node inserts the newcomer into its active view and propagates a `FORWARDJOIN` message through the overlay using a random walk controlled by a time-to-live parameter. Intermediate nodes may insert the new node into their passive view, while the final node in the walk inserts it into its active view. This mechanism helps distribute knowledge of the joining node across the network while keeping active views small.

When an active connection fails, the protocol immediately tries to replace the failed node using an entry from the passive view. This is one of the central design goals of HyParView and the main reason it is expected to perform better under churn. The passive view itself is refreshed periodically through shuffle messages that exchange neighbor information across the overlay.

3.2.3 Why Compare Cyclon and HyParView. These two membership protocols were implemented because they represent different philosophies of overlay maintenance. Cyclon relies on a single randomized structure maintained through periodic exchanges, while HyParView relies on a small communication view plus a larger backup view, combining reactive and periodic mechanisms. By implementing both, we can observe how broadcast protocols behave when the overlay is either simpler and lighter or more explicitly designed for fault recovery.

3.3 Broadcast Layer

3.3.1 Flood Broadcast. The flood broadcast implementation is the simplest dissemination mechanism in the system. When a node first receives a message, it delivers it locally and forwards it to all current neighbors except, when applicable, the sender from which the message was received. To avoid infinite retransmission loops, each node stores the identifiers of messages it has already processed and discards duplicates.

This protocol serves as a baseline. Its expected strength is very high reliability in connected overlays. Its expected weakness is the large number of duplicate transmissions generated by forwarding to all neighbors.

3.3.2 Eager-Push Gossip. In eager-push gossip, a node forwards each newly received message only to a randomly selected subset of its neighbors. The size of this subset is the fanout. As in flood broadcast, duplicate messages are ignored through the use of message identifiers.

Compared with flooding, this protocol introduces an explicit trade-off. Forwarding to fewer neighbors reduces overhead, but also makes dissemination depend more strongly on the quality of the overlay and on the selected fanout. This protocol is therefore useful to study the cost of probabilistic dissemination under different membership layers.

3.3.3 Hybrid Adaptive Gossip. The custom protocol developed for this project combines eager and lazy dissemination. When a node first receives a message, it divides the available neighbors into two groups. A subset of neighbors receives the full message immediately and forms the eager set. The remaining selected neighbors receive only a lightweight notification containing the message identifier and form the lazy set.

If a node receives a lazy notification for a message it does not yet have, it sends an explicit request to obtain the full payload. This design avoids sending the full message to every target immediately and reduces unnecessary transmissions when the message has already reached a node through another path.

The protocol also includes an adaptive mechanism based on duplicate observations. Each node tracks how many first-time and duplicate full messages it receives over a period of time. When the observed duplicate rate is high, the eager fanout is reduced. When the duplicate rate is low, the eager fanout is increased, within predefined bounds. The goal is to adapt dissemination aggressiveness to the redundancy currently observed in the network.

3.4 Message Processing and Deduplication

All broadcast protocols in our system use message identifiers to ensure that each application message is delivered at most once per node. When a message is received for the first time, it is stored in the local cache, delivered to the application, and processed according to the semantics of the selected broadcast protocol. If the same message is received again, it is classified as a duplicate and discarded, although in the hybrid protocol duplicate observations are also used as input for the adaptive fanout mechanism.

This deduplication mechanism is fundamental for correctness. Without it, both flood and gossip-based protocols would repeatedly retransmit the same content, leading to unnecessary work and potentially unbounded message circulation.

3.5 Correctness Discussion

At a high level, the correctness of the implementation depends on two main properties. First, the membership layer must maintain a connected enough overlay so that broadcast can progress through the network. Second, the broadcast layer must guarantee that a message is delivered at most once at each node while still being forwarded according to the rules of the protocol.

The first property is supported by the design of Cyclon and HyParView, which continuously refresh neighbor information and attempt to avoid overlay fragmentation. The second property is guaranteed by the use of unique message identifiers and local caches of already seen messages.

For the hybrid protocol, correctness additionally depends on the interaction between lazy notifications and explicit requests. A lazy notification never delivers a message by itself; it only informs a node that the message exists. Delivery occurs only when the full payload is received, either eagerly or as a reply to a request. This preserves the same at-most-once delivery semantics as the other broadcast protocols.

3.6 Implementation Summary

In summary, the implementation was structured to support a clean comparison between overlay construction and dissemination strategies. Cyclon and HyParView provide two distinct membership mechanisms, while flooding, eager-push gossip, and the proposed hybrid adaptive protocol provide three dissemination strategies with different reliability and overhead trade-offs. This modular structure also supports the experimental methodology presented later in the paper.

4 Experimental Evaluation

5 Conclusions

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.